DAY 01 · HINDU COLLEGE AMRITSAR · BCA SEMESTER III

Orientation & Setup.

Welcome to a 24-day internship where you ship something real — built entirely in Python.

PROGRAM

Python Internship

LED BY

Zlaark

SESSION

1 of 24

AGENDA · DAY 01

How we'll spend our first 90 minutes.

A working setup beats a perfect setup. Today we install together, so no one's left behind.

DURATION
90 min

00:00 - 00:10

Welcome & Intros

Who's in the room and what we're building over 24 days.

00:10 - 00:25

The 24-Day Roadmap

Five phases, what you'll ship by Demo Day, and the rules of engagement.

00:25 - 00:35

Pre-Install Checklist

What every laptop needs before we touch the installers.

00:35 - 00:55

Install Together

Python, VS Code, Git, a virtual environment, and Streamlit — one step at a time.

00:55 - 01:15

Hello, Streamlit

Your first app: five lines of Python, opens in a browser, no HTML anywhere.

01:15 - 01:30

Q&A and What's Next

Open floor, then a preview of Day 2.

INTRODUCTIONS · 10 MIN

One person runs the whole program.

Not passed between trainers. The same person sits with every group through the build weeks.

Zlaark

Instructor & Program Designer

- Built this curriculum from scratch for Hindu College Amritsar.
- Will personally sit with every group across all 24 days.
- Reachable throughout the build weeks, not just during sessions.

hello@zlaark.com · www.zlaark.com

ROUND OF INTROS

90 seconds each. Three things:

1. Your name and what you go by
2. One Python thing you've built or tried
3. One thing you want out of these 24 days

There are no wrong answers. Honest starting points help me pace the room.

THE PROGRAM · AT A GLANCE

Five phases. Twenty-four days. One working app.

Every session earns its place because the group project needs it that week.

YOU ARE HERE

01

Python Refresher & Tooling

DAYS 1–3

Set up, clean Python habits, Git as a habit from day one.

02

Building Interfaces in Streamlit

DAYS 4–8

Plain Python into an app people can click through. No HTML, no CSS.

03

Data, Storage & A First Taste of AI

DAYS 9–14

Files, SQLite, live APIs, and TF-IDF similarity behind resume matchers.

04

Guided Group Project

DAYS 15–21

Groups of 4–5. Five sprints. Instructor moves between teams.

05

Ship & Present

DAYS 22–24

Deploy to Streamlit Cloud, dress rehearsal, Demo Day in front of faculty.

TODAY → Day 1 of 24 → Day 24 of 24 ← DEMO DAY

BY DAY 24 · OUTCOMES

Open a blank file. Build a working app. Explain every line.

That's the bar. Not a printed report, not a screenshot of someone else's code.

24
days of guided, on-campus building

4–5
students per group, mixed by ability

1
live app, demoed to a faculty panel

Every student will be able to

- Write clean, well-organised Python beyond textbook problems
- Build a working app interface using Streamlit
- Work with real data — files, spreadsheets, a simple database
- Use a basic AI technique like text similarity inside a real project
- Use Git and GitHub the way an actual team does

How you'll be assessed

- Daily attendance and participation (min 80% to qualify for certificate)
- The Day 8 solo challenge, before groups are formed
- How well the finished app matches the scope locked on Day 16
- Actual use of Git — regular, meaningful commits
- The final presentation — every student defends their own part

BEFORE WE INSTALL · 10 MIN

Checklist before we touch any installer.

If anything below is missing, flag it now. We fix it together before moving on.

HARDWARE & OS

- [ ] **A laptop you can bring every session**
  Shared lab machines work, but a personal laptop means you keep the setup.
- [ ] **Windows 10/11, macOS, or Ubuntu**
  All three are fine. We'll cover Windows quirks live; macOS/Linux users follow along.
- [ ] **At least 5 GB free disk space**
  Python + VS Code + Git + Streamlit together need about 2 GB, leave room for project data.
- [ ] **A reliable internet connection**
  We're downloading installers together. A phone hotspot works as backup.

ACCOUNTS & ACCESS

- [ ] **A working email address**
  Needed for GitHub and Streamlit Cloud signups later.
- [ ] **Admin rights on your laptop**
  Required to install Python and Git system-wide. Lab machines: ask IT now.
- [ ] **A GitHub account (free)**
  We'll create one together on Day 3 if you don't have one. Pre-create it today if you can.
- [ ] **VS Code extensions readiness**
  No action yet, just be ready to install the Python extension in class.

Stuck on any of these? Raise your hand now.
We sort it before the install block — together, out loud, no one stuck silently.

INDUSTRY · WHY THIS MATTERS

6 real-world deployments

Python isn’t a college subject. It’s what runs the apps you use.

A quick look at where the language you'll write today actually shows up in real jobs.

01

Netflix

RECOMMENDATION ENGINE

Python scripts decide what show you see next based on what you watched. Pandas + scikit-learn, the same libraries you'll meet on Day 12.

02

Instagram

BACKEND SERVICES

Parts of the photo pipeline run on Django, a Python web framework. Every upload touches Python before it lands on your feed.

03

NASA

MISSION DATA & OPEN-SOURCE

Python processes telemetry and powers open-source tools like astropy. Used in real spacecraft operations, not just tutorials.

04

Spotify

AUDIO FEATURES & DISCOVERY

Python pipelines extract audio features that power the Discover Weekly playlist you've probably heard this week.

05

Research & Science

DATA ANALYSIS AT SCALE

Pandas, NumPy and Jupyter are the lingua franca of modern research — from genomics to climate models at the IPCC.

06

AI / ML industry

WHERE EVERY MODEL STARTS

TensorFlow, PyTorch, Hugging Face — every major AI framework has a Python-first API. The model behind ChatGPT was trained with Python tooling.

YOUR DAY-24 PROJECT uses the same families of tools — **scikit-learn** for similarity, **pandas** for data, **Streamlit** for the interface — at a beginner scale.

python.org

01 Python 3.12+ on every laptop.

We install the official Python installer together. Nobody moves on until `python --version` works on every machine.

WHAT TO DO

1 Download

Go to **python.org** → Downloads → grab the installer for your OS. The site auto-detects Windows / macOS.

2 Run the installer

Windows: tick **"Add Python to PATH"** BEFORE clicking Install. This is the #1 thing students forget.

3 Verify install

Open a **fresh** terminal (Terminal on macOS, PowerShell on Windows). Do NOT reuse an already-open one.

4 Move on when green

You see a version number that starts with `3.12` or higher. Raise your hand if you see `"command not found"`.

```
# Windows users: open PowerShell, NOT cmd
# macOS / Linux users: open Terminal

$ python --version
Python 3.12.3

$ pip --version
pip 24.0 from ... (python 3.12)

# If you see "python: command not found":
#   → Windows: reinstall, tick "Add to PATH"
#   → macOS: try "python3 --version" instead
```

⚠ **WINDOWS QUIRK** · If `python` opens the Microsoft Store, run `python3` instead, or fix PATH in **Settings → Environment Variables**.

code.visualstudio.com

02 VS Code — the editor we'll live in for 24 days.

Free, Microsoft-built, and the most-used Python editor in the industry. Not the only option, but the one we standardize on.

WHAT TO DO

1 Download

code.visualstudio.com → Download for your OS. Windows / macOS / Linux installers are all on the front page.

2 Install & launch

Accept the defaults. On Windows, tick **"Add to PATH"**. Launch once it's done.

3 Install the Python extension

Open VS Code → click the **Extensions** icon (left sidebar, looks like 4 squares) → search `"Python"` → install the one by Microsoft.

Python — by Microsoft — REQUIRED

Required. Adds linting, autocomplete, run buttons, debugging.

Pylance — by Microsoft — AUTO

Bundled with the Python extension. Faster IntelliSense.

GitLens — by GitKraken — OPTIONAL

Optional. We'll come back to this on Day 3.

VERIFY IT WORKS — Create a file called `hello.py`, type `print("hi")`, click the Play button in the top-right. You should see `hi` in the terminal panel below.

03 Git — so your code remembers everything.

We use it as a team habit from Day 3. Today we just install and configure your identity.

WHAT TO DO

1 Download & install

git-scm.com → Downloads → pick your OS. Windows users: accept the default options, including **"Git from the command line"**.

2 Verify install

Open a fresh terminal and run `git --version`. You should see something like `git version 2.43.0`.

3 Configure your identity

Tell Git who you are. This name and email go on **every commit you make for the next 24 days**.

```
# Set the name and email that goes on every commit
$ git config --global user.name "Your Real Name"
$ git config --global user.email "you@example.com"

# Verify
$ git config --global user.name
Your Real Name

# Tip: use the same email as your GitHub account
```

GITHUB ACCOUNT

- If you don't have one yet: go to **github.com** → Sign up → use the same email you just configured.
- Verify with: `gh auth login` (optional — we'll cover the GitHub CLI on Day 3).
- **Today's goal:** account exists, email matches your Git config.

A virtual environment keeps your laptop clean.

A Python project gets its own little folder of dependencies. Your system Python stays untouched. This is how real teams work — start the habit on Day 1.

WHY BOTHER

No version clashes

Project A needs Streamlit 1.30, Project B needs 1.40. Without a venv, one breaks the other. With one, each project has its own.

Easy to throw away

Something breaks? Just delete the venv folder and recreate it. Your code is untouched. **Five seconds to a clean slate.**

Team-friendly

You share a `requirements.txt` of what's inside your venv. A teammate recreates the exact same environment on their laptop.

```
# 1. Create a venv named .venv in your project folder
$ python -m venv .venv

# 2. Activate it — do this EVERY session
# Windows (PowerShell):
$ .venv\Scripts\Activate.ps1

# macOS / Linux:
$ source .venv/bin/activate

# 3. You should now see (.venv) in your prompt:
(.venv) $

# 4. Install things INTO the venv, not globally:
(.venv) $ pip install streamlit

# 5. Leave the venv when done:
```

TIP Add `.venv/` to your `.gitignore` from Day 1. **Never commit a venv folder.**

FINAL INSTALL

04 Streamlit — Python that opens in a browser.

No HTML. No CSS. No JavaScript. Just Python functions that turn into buttons, sliders and pages.

WHAT TO DO

1 Activate your venv first

Make sure you see `(.venv)` in your terminal prompt. If you don't, Streamlit installs **globally** and that's a mess to undo.

2 Install Streamlit

Run `pip install streamlit`. About 30 seconds. Pulls in **pandas**, **numpy**, and a handful of helpers automatically.

3 Verify with the built-in demo

Run `streamlit hello`. A browser tab opens with a demo app. If you see it, **you're done with installs for today.**

```
# Make sure your venv is active!
(.venv) $ pip install streamlit

# Watch it pull dependencies...
Successfully installed streamlit-1.40.0 ...

# Sanity check the version
(.venv) $ streamlit --version
Streamlit, version 1.40.0

# Open the built-in demo app
(.venv) $ streamlit hello

# → A browser tab opens at http://localhost:8501
# → You should see a welcome page with a sidebar of demos
```

IF THE BROWSER DOESN'T OPEN — Manually go to `http://localhost:8501` in any browser. The Streamlit server runs in your terminal; **closing the terminal stops the app.**

YOUR FIRST APP · 5 LINES OF PYTHON

A Streamlit app is just a Python file that runs top to bottom.

No boilerplate. No framework ceremony. Save this as `app.py`, run `streamlit run app.py`, watch a browser open.

app.py PYTHON · STREAMLIT

```python
import streamlit as st

st.title("Hello, Hindu College Amritsar.")
name = st.text_input("What's your name?")

if name:
    st.write(f"Welcome to Day 1, {name}.")
    st.balloons()
```

LINE BY LINE

1 `import streamlit as st`
The library we just installed. Convention: alias it as `st`.

3 `st.title(...)`
Renders an H1 heading on the page. Plain Python function call.

4 `st.text_input(...)`
Renders a text box. Whatever the user types gets returned and stored in `name`.

6 `if name:`
Standard Python. Runs only after the user types something.

7 `st.write(f"...")`
Writes anything to the page — text, dataframes, charts. The f-string injects the name.

8 `st.balloons()`
A small celebration animation. Because Day 1 deserves it.

KEY IDEA — Every interaction re-runs the entire file top to bottom. That's the Streamlit model. We'll go deeper on Day 4.

RUN IT · SEE IT IN A BROWSER

Three lines in the terminal. One browser tab. You've built an app.

If this works, every tool you need for the next 24 days works. That's the checkpoint for today.

```
# From the folder that contains app.py, with your venv active:
(.venv) $ streamlit run app.py

  You can now view your Streamlit app in your browser.
  Local URL:   http://localhost:8501
  Network URL: http://192.168.x.x:8501
```

YOU SHOULD SEE

- ✓ A browser tab opens automatically at `localhost:8501`
- ✓ The page title `"Hello, Hindu College Amritsar."`
- ✓ A text box asking `"What's your name?"`
- ✓ Typing a name + pressing Enter shows a welcome line
- ✓ Balloons animate across the screen

IF IT BROKE

- ✕ `command not found: streamlit` → venv not active. Run `source .venv/bin/activate` (macOS) or `.venv\Scripts\Activate.ps1` (Windows).
- ✕ `ModuleNotFoundError: No module named streamlit` → install forgot the venv. Run `pip install streamlit` again with venv active.
- ✕ Browser didn't open → go to `http://localhost:8501` manually.
- ✕ Port 8501 already in use → `streamlit run app.py --server.port 8502`.

CHECKPOINT FOR TODAY ✓ — Every laptop can run a Streamlit hello-world app and open it in a browser. If yours doesn't, we fix it together before you leave.

NEXT SESSION · DAY 02

Python, Properly.

Today was setup. Tomorrow we write code that earns its place in a real project.

Day 02 TOMORROW'S SESSION

Python, Properly

Goal — Get everyone's Python back to a level where writing a function or reading a file feels automatic, not something to look up.

Functions with real inputs and outputs

Not just `print` statements. Functions that return things other code can use.

Reading and writing files

Every later project needs to save something. We start the habit here.

Clean code habits

The things that matter once you're building something real instead of a one-off exercise.

By the End — Every student has written and run a function that reads a file and returns something useful from it.

BEFORE NEXT SESSION

Four small things. None should take more than ten minutes.

- [ ] **Play with your app.py**
  Change the strings, add a second `st.text_input`, see what happens.
- [ ] **Open VS Code, make a .py file**
  Just `print("hello")`. Get used to the editor + run button workflow.
- [ ] **Skim docs.python.org/3/tutorial**
  Sections 4 (Control Flow) and 7 (Input and Output). Five minutes each.
- [ ] **Bring questions**
  Anything shaky from today, we open Day 2 with a 5-min Q&A.

Q&A · CLOSING · 15 MIN

Questions are the point.

If something didn't click today, now is the time. Tomorrow builds on today — we close gaps before they widen.

OPEN THE FLOOR

Ask Now

- ✓ "Which step broke for you?"
- ✓ "What's a Python thing you want to be solid by Day 24?"
- ✓ "Anything from the 24-day roadmap that felt unclear?"

BETWEEN SESSIONS

Get Unstuck

- ✓ **Email · `hello@zlaark.com`**
  For longer questions, code snippets welcome.
- ✓ **Web · `www.zlaark.com`**
  Program updates and resources.
- ✓ **In-session · raise a hand**
  No question is too small. We move at the room's pace.

RULES OF ENGAGEMENT

How We Work

- ✓ Min 80% attendance to qualify for the certificate.
- ✓ Daily commits to Git from Day 3 onward.
- ✓ Demo Day is mandatory — every student presents.

See you on Day 2.